

Zusammenfassung – Aufgabe 2:

Spielmodell & Spiellogik (LockSnake)

Aufgabe 2 bestand darin, das Spielmodell von *LockSnake* so zu erweitern, dass das Spiel vollständig spielbar wird. Dazu mussten zwei zentrale Komponenten implementiert werden:

- **GameState** – Modellierung des gesamten Spiellogik-Zustands
- **GameEngine** – Steuerung des Spiels, Verbindung zwischen Logik und UI

Das Ziel war, dass das Spiel korrekt läuft, Eingaben verarbeitet und die Pin-Mechanik funktioniert.

1. GameState – Modellierung des Spielzustands

Der **GameState** repräsentiert den kompletten Zustand des Spiels zu einem bestimmten Zeitpunkt.

Er ist immutable, d. h. jede Änderung erzeugt einen neuen GameState.

1.1 Bestandteile des GameState

Der GameState enthält:

- das **Level** (Spielfeld, Wände, Startposition)
- die **Schlange** (Positionen aller Segmente)
- die **Pins** (LOW/HIGH, Aktivierungsrichtung)
- den **Spielstatus** (RUNNING, WON, LOST_...)
- die **aktuelle Blickrichtung** (pendingDirection)

1.2 tick() – zentrale Spiellogik

Die Methode **tick()** berechnet den nächsten Spielzustand.

Sie führt folgende Schritte aus:

1. **Spiel läuft nicht** → keine Änderung
2. **Keine Blickrichtung** → keine Bewegung
3. **Berechnung der nächsten Kopfposition**
4. **Out-of-Bounds prüfen**
5. **Wandkollision prüfen**
6. **Selbstkollision prüfen**
7. **Pin an der nächsten Position finden**
8. **Pin-Mechanik**

- falsche Richtung oder HIGH → blockiert
 - LOW + richtige Richtung → Pin wird HIGH
 - Schlange bleibt beim Aktivieren stehen
9. **Normale Bewegung**
- Schlange wächst in Blickrichtung
10. **Gewinnbedingung prüfen**
- alle Pins HIGH → Status = WON

2. GameEngine – Steuerung des Spiels

Die **GameEngine** ist das Bindeglied zwischen:

- **GameState** (Logik)
- **GamePanel** (UI)
- **Tastatureingaben**
- **Timer (Spiel-Tick)**

2.1 Aufgaben der GameEngine

A) Startzustand erzeugen

- Schlange aus Level-Startposition
- Pins aus dem Level
- Status = RUNNING
- Blickrichtung = NONE

B) update(Direction)

Wird vom UI aufgerufen, wenn der Spieler eine Richtungstaste drückt.

- neue Blickrichtung setzen
- neuen GameState erzeugen
- UI benachrichtigen

C) tick()

Wird vom Timer aufgerufen.

- GameState.tick() ausführen
- UI benachrichtigen

D) Observer-Pattern

Die Engine verwaltet eine Liste von Observern (z. B. GamePanel).

Nach jeder Änderung wird **notifyObservers()** aufgerufen.

3. Zusammenspiel der Komponenten

3.1 Main

- lädt das Level aus `/levels/level1.txt`
- erzeugt GameEngine und GamePanel
- verbindet Engine ↔ Panel
- startet einen Swing-Timer, der regelmäßig `engine.tick()` aufruft

3.2 GamePanel

- zeigt das Spielfeld an
- empfängt Tastatureingaben
- ruft `engine.update(direction)` auf
- wird durch Observer-Mechanismus aktualisiert